

Generative AI-Driven Polymorphic Malware: Threats, Detection, and Governance

Amalachukwu Azubike, Bereni Iyagba, Alfred Amoah, Kokou Houmey

CIS*6590 Graduate Seminar

Master of Cybersecurity and Threat Intelligence

University of Guelph, Ontario, Canada

Abstract

Generative artificial intelligence can automate malware code generation, mutation, and evasive optimization, potentially amplifying long-standing polymorphic and metamorphic techniques. This structured literature review synthesizes 55 verified sources spanning adversarial malware, large-language-model-driven mutation, static, behavioral, and graph-based detection, robustness, explainability, benchmarking, and AI governance. The review finds emerging evidence that generative AI lowers the effort required to produce structurally diverse and function-preserving variants, but direct evidence of systematic evasion of modern production endpoint detection and response systems remains limited. Existing detectors are frequently evaluated on legacy or laboratory datasets that may not capture temporal drift, endpoint conditions, or environment-aware adaptation. A two-axis taxonomy is proposed based on mutation depth and generative control paradigm. The paper recommends generative-AI-specific benchmarks, layered representation-diverse detection, adaptive and endpoint-oriented evaluation, trustworthy analyst-facing explanations, and measurable multilevel governance.

Keywords: Generative artificial intelligence; polymorphic malware; metamorphic malware; large language models; adversarial machine learning; malware detection; explainable artificial intelligence; AI governance

I. INTRODUCTION

Polymorphism has long been used by malicious software to evade detection by changing its observable representation while preserving its intended functionality. Traditional polymorphic malware commonly modifies encrypted payloads, decryptors, packing configurations, or other surface features, whereas metamorphic malware rewrites portions of its own implementation through techniques such as instruction substitution, register reassignment, code transposition, and insertion of nonfunctional instructions [1], [2]. Although the distinction is sometimes blurred in the literature, both approaches exploit a fundamental weakness in malware defenses that depend heavily on stable representations: a malicious program may remain functionally equivalent even when its bytes, instructions, control-flow structures, or extracted features differ substantially across variants.

The growth of machine-learning-based malware detection did not eliminate this problem. Instead, it expanded the target of evasion from manually authored signatures to statistical decision boundaries. Research in adversarial machine learning has shown that carefully constructed input modifications can cause classifiers to produce incorrect predictions while preserving the attacker's operational objective [3], [4]. Malware-specific studies have further demonstrated that polymorphic transformations and adversarial modifications can deceive artificial-intelligence-based detectors [5], [6]. However, not every adversarial malware example constitutes polymorphic malware. Feature-space perturbations that cannot be reconstructed as valid executables; ordinary code obfuscation, non-functional generated code, and function-preserving executable mutation represent different technical constructs. Maintaining these distinctions is essential because they determine whether an attack is operationally feasible and whether a proposed defense has been evaluated against a realistic threat.

Generative artificial intelligence introduces a potentially important shift in this attack-defense relationship. Large language models can generate, rewrite, translate, and explain source code, while generative adversarial networks and reinforcement-learning systems can optimize transformations against detector feedback. Recent studies have examined large-language-model-assisted assembly obfuscation, automated malware-development capabilities, fine-tuned code mutation, and the generation of structurally diverse offensive code [7]-[11]. Collectively, this emerging evidence suggests that generative models may reduce the expertise, time, and manual effort required to produce diverse malware variants. They may also support iterative workflows in which variants are generated, tested, and revised against static analyzers, sandboxes, or machine-learning classifiers.

Figure 1 summarizes the evolution of automated malware mutations. The progression reflects increasing automation, transformation flexibility, and environmental feedback rather than the replacement of earlier techniques; rule-based, adversarial optimized, LLM-driven, and feedback-based approaches may coexist within the same attack workflow.

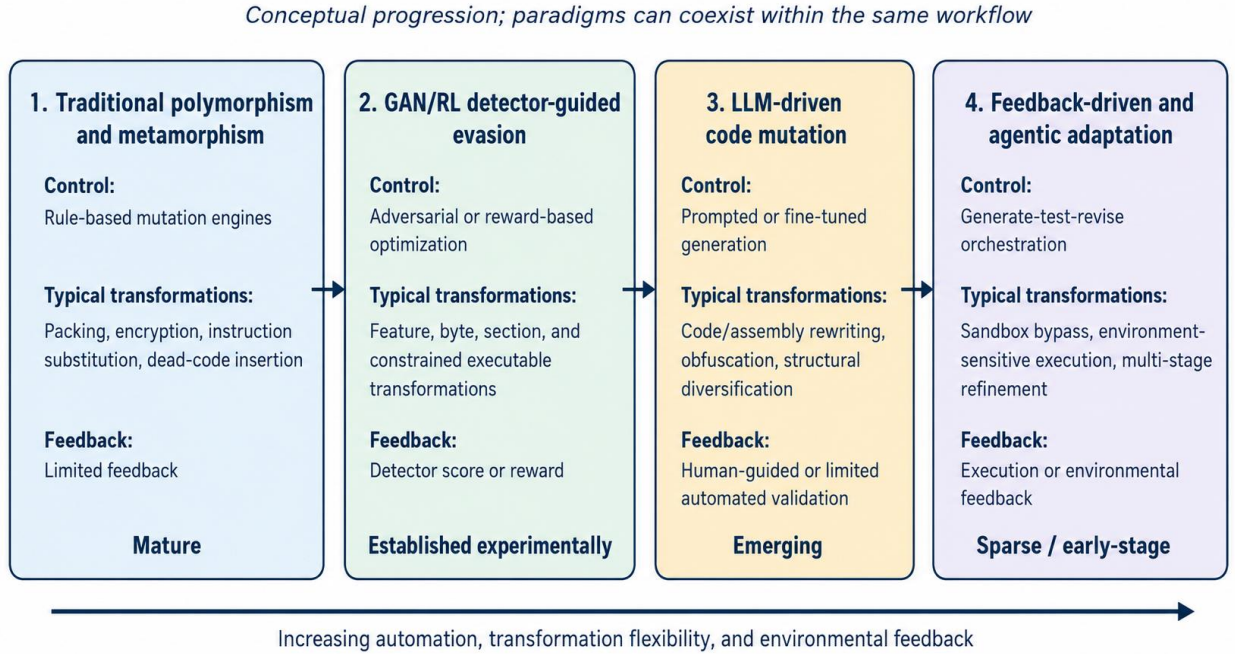


Fig. 1. Evolution of malware-mutation paradigms from traditional polymorphism and metamorphism to adversarially optimized, large-language-model-driven, and feedback-based adaptive workflows.

Nevertheless, the current evidence does not establish that generative-AI-mutated malware consistently defeats modern production endpoint detection and response systems at scale. Much of the most direct literature consists of controlled capability studies, narrowly scoped experiments, or recent preprints. Some studies evaluate source-code fragments, scripts, feature vectors, or laboratory-generated binaries rather than complete attack chains deployed against operational enterprise defenses. The central concern is therefore not that generative AI has already rendered malware detection ineffective, but that it may increase the speed, diversity, and adaptability of malware generation faster than evaluation practices and defensive systems can be updated.

This concern is amplified by limitations in the defensive evidence base. Static detection remains efficient and operationally valuable, but it is susceptible to packing, obfuscation, representation changes, and adversarial manipulation. Behavioral, semantic, graph-based, and explainable detection methods seek to capture properties that are more difficult to modify without affecting malicious functionality. Yet many such systems have been evaluated on datasets and experimental conditions developed before the emergence of large-scale generative code models. Their reported performance may therefore provide limited evidence about resilience to generative-AI-driven mutation, environment-aware adaptation, concept drift, or sandbox-to-endpoint transfer. At the same time, existing AI governance instruments and threat-modeling frameworks provide broad mechanisms for managing dual-use risks, but they rarely define malware-specific controls or measurable requirements for evaluating model-assisted malware generation [12], [13].

Against this background, this paper addresses the following main research question:

How is generative AI reshaping the generation and adaptation of polymorphic malware, how well do current detection approaches withstand these variants, and which governance controls are most relevant to the resulting dual-use risk?

Four supporting questions guide the review. First, what generative mechanisms produce function-preserving structural or behavioral diversity, and what empirical evidence demonstrates their effectiveness? Second, how robust are static, behavioral, graph-based, and explainable detection approaches against AI-generated or AI-mutated malware? Third, which datasets, temporal evaluation methods, and benchmarks are required for credible assessment under concept drift and generative mutation? Fourth, which provider, organization, and ecosystem-level governance controls can reduce dual-use malware-generation risk, and where do current frameworks remain too general?

The scope is deliberately limited to generative-AI-assisted malware generation, mutation, obfuscation, evasion, behavioral adaptation, detection, evaluation, and governance. Prompt injection, phishing, deepfakes, misinformation, model poisoning, and other AI-security issues are excluded unless they directly inform malware-generation or dual-use governance considerations. The paper adopts a structured literature review with systematic search and screening elements rather than claiming an exhaustive systematic review.

This paper makes three principal contributions. First, it distinguishes generative-AI-driven polymorphism from adjacent concepts such as feature-space adversarial examples, conventional obfuscation, and unvalidated generated code. Second, it introduces a two-axis taxonomy that classifies the literature according to mutation depth from syntactic change to environment-aware adaptation, and generative control paradigm, ranging from rule-based transformation and adversarial optimization to language-model-driven and feedback-based orchestration. Third, it maps these threat categories to detection, evaluation, and governance responses, identifying where current evidence is mature, where conclusions remain provisional, and where further research is needed.

The remainder of the paper is organized as follows. Section II explains the operational and societal motivation for examining generative AI-driven polymorphic malware. Section III synthesizes related work on traditional polymorphism, adversarial malware generation, large-language-model-assisted mutation, detection approaches, explainability, and governance. Section IV describes the structured review methodology and taxonomy development process. Section V analyzes the principal technical, evaluation, operational, and governance challenges. Section VI presents evidence-based recommendations for researchers, cybersecurity practitioners, organizations, and model providers. Section VII concludes the paper and identifies priorities for future research.

II. MOTIVATION

A. Operational and Threat Relevance

The security significance of generative AI-driven polymorphic malware lies less in the immediate appearance of an entirely new malware class than in the possibility that generative models can reduce the cost, expertise, and time required to produce evasive variants. Traditional polymorphic and metamorphic techniques often require specialized knowledge of executable formats, assembly instructions, program semantics, packing, and anti-analysis mechanisms. Generative models may partially automate these activities by producing alternative implementations, suggesting obfuscation strategies, rewriting code, or supporting iterative testing and refinement. Controlled studies of large language models have demonstrated capabilities relevant to malware development and code mutation, although their outputs vary in functional quality and operational realism [8]-[11].

This shift affects the economics of malware development. When variant production is expensive, attackers must balance the value of evasion against the engineering effort required to create and validate each transformation. Generative systems may reduce this marginal cost by rapidly producing multiple candidate implementations. Even when only a subset of generated variants compiles, executes correctly, or preserves malicious functionality, automated generation can increase the number of variants available for testing. The resulting concern is not simply that artificial intelligence can write malware, but that it may support a faster generate-test-revise cycle in which structural diversity becomes comparatively inexpensive.

The available threat evidence should nevertheless be interpreted cautiously. Peer-reviewed capability studies and emerging preprints provide stronger evidence of technical possibility than widespread operational adoption. Industry reporting has documented the use of generative-AI services by state-affiliated and criminal actors for activities such as reconnaissance, scripting, translation, vulnerability research, and content generation, but such reporting does not demonstrate that autonomous, generative-AI-driven polymorphic malware has become commonplace [14]. Similarly, contemporary threat-landscape reporting identifies artificial intelligence as an increasingly relevant enabler within the broader cyber-threat environment, while the degree of direct malware automation remains difficult to measure independently [13].

For security operations center teams, malware analysts, and incident responders, even partial automation can create operational pressure. Static indicators such as hashes, byte signatures, and fixed structural patterns become less durable when adversaries can generate numerous semantically related variants. Analysts may face clusters of samples that appear different at the representation level while sharing behavior, infrastructure, or operational purposes. This can increase reverse-engineering workload, complicate malware-family attribution, reduce the useful lifetime of detection rules, and amplify alert volume. Generative mutation, therefore, matters operationally even before fully autonomous malware generation becomes technically reliable.

B. Detection and Governance Stakes

The issue is also important because much of the malware-detection evidence base was created before generative code models became widely available. EMBER and SOREL-20M provide valuable large-scale benchmarks for static Portable Executable malware detection [15], [16]. However, these datasets were not designed specifically to represent large populations of language-model-generated, function-preserving variants. Strong performance on established benchmarks, therefore, does not automatically demonstrate robustness against generative mutation.

Evaluation methodology further complicates interpretation. For instance, TESSERACT showed that spatial and temporal biases can substantially inflate malware-classification results when experimental datasets do not reflect realistic deployment conditions [17]. Later work on concept drift similarly demonstrates that malware distributions evolve, and that models trained on historical data can lose effectiveness as attacker behavior changes. Generative systems could intensify this problem by accelerating the production of structurally novel variants, making chronological evaluation and continual reassessment increasingly important.

Behavioral detection appears promising because runtime actions, application programming interface sequences, process relationships, file-system modifications, and communication patterns are more closely connected to malicious objectives than superficial binary structure. Yet behavioral approaches introduce their own deployment challenges. Evidence from endpoint-focused research suggests that models trained primarily on sandbox observations may perform differently when transferred to real endpoints because the execution context, telemetry, background activity, and adversarial conditions change [18]. Dynamic analysis can also be affected by delayed execution, environment checks, incomplete code coverage, and resource overhead. The practical motivation is therefore not to replace static detection with behavior alone, but to determine how multiple representations can be combined and evaluated realistically.

Generative-AI-driven malware also raises governance questions because the same models and development tools may support legitimate software engineering, defensive analysis, vulnerability research, and malicious code generation. General-purpose governance frameworks such as the National Institute of Standards and Technology Artificial Intelligence Risk Management Framework, its Generative Artificial Intelligence Profile, ISO/IEC 42001, and the European Union Artificial Intelligence Act provide structures for accountability, risk assessment, documentation, monitoring, and organizational oversight [19]–[22]. Guidance on dual-use foundation-model misuse further emphasizes evaluation, access controls, incident response, and risk-management practices [23].

These instruments are relevant but not specifically designed around malware polymorphism. They generally do not prescribe how a model provider should measure code-mutation capability, how an organization should validate a model against malware-generation abuse cases, or how the cybersecurity community should benchmark AI-generated variants. This gap motivates the governance component of the paper. Effective response will require controls at multiple levels: providers must evaluate and monitor misuse capabilities; deploying organizations must restrict and log sensitive model use; and the wider ecosystem must improve benchmarks, incident sharing, and threat intelligence.

The topic is therefore practically important because it links three developments that are often studied separately: automated malware mutation, the limits of current detection evaluation, and the governance of dual-use generative models. Examining them together helps clarify which risks are already supported by evidence, which remain emerging, and which defensive investments are most likely to remain valuable as generative technologies evolve.

III. RELATED WORK

The literature relevant to generative-AI-driven polymorphic malware spans several research communities that often use different terminology and evaluation assumptions. Traditional malware research focuses on polymorphism, metamorphism, packing, and code obfuscation. Adversarial-machine-learning research examines how attackers manipulate classifier inputs or decision boundaries. More recent work studies on generative adversarial networks, reinforcement learning, and large language models as mechanisms for automating malware transformation or evasion. Defensive research, meanwhile, has progressed from signatures and static features toward behavioral, semantic, graph-based, robust, and explainable detection.

These bodies of work are related but should not be treated as equivalent. A feature-space perturbation that changes a classifier's prediction is not necessarily a valid executable mutation. Similarly, obfuscated code is not necessarily polymorphic, and generated code that fails to compile or preserve malicious behavior does not constitute operational malware. Table I establishes the terminology used throughout this review.

TABLE I. TERMINOLOGY AND CONSTRUCT BOUNDARIES

Construct	Defining characteristic	Required validity condition	Principal evaluation concern
Polymorphic malware	Changes observable representation while preserving core malicious functionality	Valid executable or script retaining intended behavior	Surface diversity may be confused with behavioral diversity
Metamorphic malware	Rewrites internal implementation rather than relying only on packing or encryption	Semantic or functional equivalence	Equivalence is difficult to demonstrate
Obfuscation	Makes code or behavior harder to inspect or interpret	May or may not preserve complete functionality	Obfuscation alone does not establish polymorphism
Adversarial malware example	Manipulates detector-relevant inputs to cause misclassification	Preferably realizable in a valid executable	Many attacks remain in feature space
AI-generated malware code	Code created or modified by a generative model	Must compile or execute and retain intended behavior	Syntactic correctness may be mistaken for operational validity
Environment-aware adaptation	Conditions execution or mutation on host, detector, or sandbox context	Requires runtime observation or feedback	Evidence remains limited and experimental

A. Traditional Polymorphism and Metamorphism

Polymorphic and metamorphic malware predate contemporary machine learning and generative artificial intelligence. Classical polymorphic malware commonly changes encrypted payloads, decryptors, packing arrangements, or other byte-level characteristics across variants. Metamorphic malware goes further by transforming the implementation itself through techniques such as instruction substitution, register reassignment, dead-code insertion, control-flow modification, and subroutine reordering [1], [2]. The objective is to reduce similarity between samples while maintaining the behavior needed to achieve the malicious goal.

Traditional literature is important for two reasons. First, it establishes that malware variation is not inherently an artificial-intelligence problem. Attackers have long sought ways to invalidate signatures and complicate reverse engineering. Generative AI changes the scale and potential automation of this process rather than inventing it.

Second, classical metamorphism provides the standard against which newer claims should be evaluated. A generative model that rewrites a small code fragment has not necessarily produced metamorphic malware unless the result remains executable and functionally equivalent. Similarly, a model that introduces superficial diversity without preserving malicious behavior may demonstrate code generation, but not operational polymorphism.

Empirical research has also shown that polymorphic transformations can affect artificial-intelligence-based malware detectors [5]. This finding connects the traditional malware literature to modern adversarial-machine-learning research: detectors trained on stable representations may be vulnerable when malicious functionality is preserved, but the representation changes. However, this does not imply that all machine-learning detectors fail equally or that every transformation remains valid across different malware families, architectures, and operating environments.

B. GAN and Reinforcement-Learning-Driven Adversarial Malware

Before large language models became widely available, generative adversarial networks and reinforcement learning had already introduced automation into malware evasion research. Early GAN-based work demonstrated that a generator could learn modifications intended to move malicious samples toward a benign region of a classifier's feature space [24]. This line of research was influential because it framed malware evasion as an optimization problem rather than a manually designed set of transformations.

Its operational limitations were equally important. Feature-space manipulation does not automatically produce a valid executable. Malware classifiers may operate on imported functions, byte histograms, section properties, application programming interface indicators, or other extracted features, but arbitrary changes to these features may not be realizable without breaking the executable or changing its behavior. This feature-to-problem-space gap became a central concern in practical adversarial-malware research [6].

Later studies attempted to close this gap by applying reinforcement learning to concrete executable transformations. These approaches typically define a set of permitted operations such as adding benign sections, modifying non-executed regions, changing metadata, or applying constrained binary transformations, and using detector feedback as a reward. Work on reinforcement-learning and GAN-assisted Windows malware mutation demonstrated that automated agents could search for combinations of transformations that evade black-box classifiers while attempting to preserve functionality [25].

AIMED-RL similarly explored reinforcement learning for constructing adversarial malware examples through executable-level transformations [26]. The contribution of this line of work was not merely improving evasion, but the introduction of an iterative decision process: an agent selects a transformation, observes the detector's response, and adjusts subsequent actions. This feedback model foreshadows later generative-AI workflows in which code is generated, tested, and revised.

MEME extended this direction through model-based reinforcement learning [27]. Rather than relying only on direct trial and error, model-based approaches attempt to learn aspects of the environment or detector response and use that learned representation to improve the mutation policy. EGAN applied evolutionary and generative methods to ransomware evasion, providing evidence that automated mutation can be adapted to specialized malware classes [28].

Query-free GAN-based evasion research has also sought to reduce dependence on repeated access to the target detector [29]. This is practically relevant because commercial malware detectors may impose rate limits, reveal little confidence information, or detect repeated probing. A locally trained surrogate model can allow the attacker to optimize transformations without continuously querying the target. However, transferability remains uncertain: an attack optimized against one surrogate may not generalize to another detector with different features or architecture.

Deep-reinforcement-learning approaches have also investigated obfuscation strategies against deep malware classifiers [30]. Across these studies, malware mutation is treated as an optimization problem constrained by executable validity, detector feedback, and transferability. These properties form the bridge from classical rule-based mutation to the LLM-driven and feedback-oriented stages summarized in Fig. 1.

C. Large-Language-Model-Driven Mutation and Adaptive Workflows

Large language models differ from earlier GAN-based malware generators because they operate over natural language, source code, assembly, scripts, and program descriptions. They can produce new implementations, explain code structure, translate between languages, suggest obfuscation strategies, and revise outputs in response to feedback. These capabilities make them plausible mutation engines, but their effectiveness depends on functional validation and realistic evaluation.

Research on large-language-model-assisted assembly obfuscation has shown that models can generate alternative assembly-code representations and apply transformations associated with obfuscation [7]. This supports the claim that language models can increase structural diversity. It does not, by itself, establish that every output is operationally

correct. Generated assembly may contain syntax errors, semantic mistakes, architecture inconsistencies, or transformations that change program behavior. Consequently, compilation and behavioral testing are essential components of any credible evaluation.

Capability studies examining ChatGPT and similar models have shown that general-purpose language models can assist with malware-development tasks under controlled conditions [8]. Such studies are valuable because they demonstrate the relevance of foundation models to offensive software development. Nevertheless, they should not be interpreted as proof that fully autonomous malware development is already common. Many demonstrations require human prompting, debugging, code integration, and correction.

Fine-tuning introduces a more specialized risk. Setak and Madani examined fine-tuned language models for code mutation, suggesting that model adaptation can increase consistency and specialization for transformation tasks [9]. Fine-tuned models may require less detailed prompting and may generate variants better aligned with a specific programming language, architecture, or evasion objective. At the same time, their performance depends heavily on training data, validation methods, and the definition of successful mutation.

Horte and Tapiador investigated polymorphism in large-language-model-generated offensive code by measuring structural diversity across generated variants [10]. Their work is particularly relevant because it moves beyond anecdotal examples and attempts to quantify mutation. However, structural diversity should not be equated automatically with functional or behavioral diversity. A set of samples may differ substantially in source or syntax while implementing nearly identical execution behavior. Conversely, small structural changes may alter functionality in ways that invalidate the sample.

Madani's work on the evolution of metamorphic malware through large language models similarly argues that language models may serve as flexible transformation engines [11]. The significance of this research lies in the possibility that mutation logic no longer needs to be encoded entirely in a fixed metamorphic engine. A generative model can select among multiple transformations, produce new implementations, and respond to contextual instructions.

More recent research extends this idea toward adaptive workflows. Sui et al. examined the use of a large language model to generate bypass rules intended to evade malware-analysis sandboxes [31]. This represents a movement from static code mutation toward environment-aware reasoning. The model is not merely changing syntax; it is generating logic based on assumptions about the analysis environment. Because this evidence is recent and preprint-based, it should be treated as an emerging capability rather than an established operational pattern.

MalGEN presents a testbed for generating and evaluating malware behaviors [32]. Its importance lies in the shift from individual code samples to repeatable behavior-generation and evaluation workflows. Testbeds of this kind can support controlled experimentation, but they also raise methodological questions: generated behavior must be isolated safely, labeled accurately, and evaluated using transparent criteria.

Research on AI-generated PowerShell malware similarly provides an experimental framework and dataset for studying generated malicious scripts [33]. PowerShell is a useful case because it supports rapid generation and execution without requiring compilation. However, script-specific results should not be generalized to native compiled malware, kernel-level malware, or complex multistage operations.

Taken together, the large-language-model literature suggests a progression from prompted code generation to prompted mutation or obfuscation, fine-tuned mutation models, execution-aware generation and testing, environment-aware bypass generation, and feedback-driven or multi-agent behavior orchestration. The empirical evidence becomes thinner as the level of adaptation increases. Prompted and fine-tuned mutation have several direct studies, whereas context-aware, runtime-adaptive, and agentic malware generation remain underrepresented. This uneven maturity is a central finding of the review.

D. Evolution of Malware Detection

Table II compares the main malware-detection families considered in this review. The approaches differ in the evidence they analyze, their operational strengths, and the mutation strategies most likely to challenge them.

TABLE II. COMPARISON OF MALWARE-DETECTION APPROACHES

Detection approach	Primary evidence	Main strength	Main limitation against GenAI mutation	Recommended role
Signature and conventional static analysis	Hashes, strings, headers, imports, and byte patterns	Fast, scalable, and operationally mature	Sensitive to packing, rewriting, and representation changes	Initial triage and known-threat detection
Static ML and deep learning	Learned byte, header, import, or semantic features	Detects patterns beyond fixed signatures	Vulnerable to adversarial manipulation, drift, and dataset bias	High-volume screening with continual validation
Dynamic and behavioral analysis	API calls, processes, files, registry, memory, and network activity	Closer to malicious operational intent	Sandbox evasion, incomplete execution, latency, and endpoint-transfer gaps	Confirmation and behavior-oriented detection
Graph and semantic analysis	Control-flow, call, dependency, and behavior graphs	Captures relationships and program structure	Graph manipulation, computational cost, and uncertain adversarial robustness	Structural and relational correlation
Robustness-oriented detection	Explicit perturbation models and adversarial tests	Provides bounded empirical or formal guarantees	Guarantees apply only to defined transformations	Model assurance and security evaluation
Explainable and analyst-centred detection	Feature attribution, behavioral evidence, and model explanations	Supports investigation and accountability	Explanations can be unstable or manipulated	Human-in-the-loop validation
Defensive LLM-assisted analysis	Code, behavioral reports, telemetry, and natural-language reasoning	Supports summarization and multi-source interpretation	Hallucination, prompt sensitivity, and limited reproducibility	Analyst assistance rather than autonomous decision-making

1) Static and Machine-Learning-Based Detection

Static malware detection analyzes a file without executing it. Traditional methods use hashes, byte signatures, imported functions, strings, headers, section structures, and known code patterns. Machine-learning approaches extend static analysis by learning statistical relationships among extracted features. EMBER and SOREL-20M provide large datasets that have enabled reproducible development and comparison of static Portable Executable malware classifiers [15], [16].

Static approaches remain operationally attractive because they are fast, scalable, and comparatively easy to deploy. Their principal limitation is dependence on observable representation. Packing, obfuscation, adversarial modification, and code mutation can alter static features without removing malicious intent.

The literature on adversarial Windows malware demonstrates that static classifiers can be manipulated through append, patch, metadata, and feature-level transformations [6]. Certified-robustness research attempts to provide formal guarantees within defined perturbation models. DRSM applies de-randomized smoothing to malware classifiers to certify robustness against specified modifications [34]. Related work has examined certified robustness against patch and append attacks [35].

These approaches improve rigor by defining what transformations the detector can tolerate. However, certification is necessarily limited to the modeled perturbation set. A classifier certified against appended bytes is not automatically robust against semantic code rewriting, runtime adaptation, or behavior changes. Certification should therefore be interpreted as a bounded guarantee rather than universal resistance to polymorphism.

A broad systematic review of artificial-intelligence-based malware detection shows the maturity and diversity of machine-learning approaches, but also highlights persistent problems involving datasets, reproducibility, interpretability, and deployment realism [36]. Robustness-by-design approaches continue to explore architectures that provide stronger guarantees against adversarial malware [43], although much of this work remains experimental.

2) Dynamic and Behavioral Detection

Dynamic detection analyzes a program during execution. Relevant signals include application programming interface calls, file-system activity, registry changes, process creation, memory behavior, network connections, and interactions among processes. Behavioral approaches are attractive because malicious objectives often require observable actions that are harder to change safely than bytes or syntax.

DMalNet combines dynamic malware analysis with application programming interface feature engineering and graph learning [37]. This approach illustrates how behavioral and relational evidence can be represented jointly. Instead of treating each event independently, a graph model can capture relationships among calls or entities.

Mauri and Damiani directly examined how behavioral classifiers can be hardened against polymorphic malware using an ensemble approach [44]. This study is especially relevant because it evaluates polymorphism at the behavioral-classifier level rather than assuming that behavioral detection is inherently immune. Behavior can also be modified, delayed, distributed, or selectively activated.

BEACON applies large-language-model embeddings and deep learning to behavioral malware classification [45]. It represents a defensive use of generative-model-derived representations: language-model embeddings are used to encode sandbox reports or behavioral descriptions rather than to generate malware.

Trident further combines large language models with behavioral and static features [46]. Although this work remains a preprint, it addresses an important research gap by investigating whether language-model reasoning and multiple feature representations can improve malware detection under evolving conditions.

Behavioral detection nevertheless faces significant deployment constraints. Sandbox execution may not expose every malicious branch. Malware may detect virtualized environments, delay execution, require user interaction, or remain dormant until a specific condition is met. Endpoint telemetry also differs from sandbox telemetry because real systems contain background activity, legitimate software interactions, and varying configurations. These factors limit the transferability of laboratory-trained behavioral models.

3) Semantic and Graph-Based Detection

Semantic detection seeks to model meaning or function rather than only surface representation. SeMalBERT applies transformer-based semantic representations to malware detection [38]. Such methods may be less sensitive to superficial changes, but their robustness depends on how semantics are represented and whether attackers can manipulate the input representation.

Graph-based methods model structural relationships within malware. Control-flow graphs represent possible execution paths, while call graphs represent relationships among functions. Dynamic graphs can represent sequences or dependencies among application programming interface calls.

Graph-transformer research has used control-flow structures for few-shot malware classification [39]. MalGraph applies hierarchical graph neural networks to Windows malware detection [40]. MGDN models malware through a graph-of-graphs structure, combining multiple relational representations [41]. These approaches are promising because they attempt to capture properties more stable than raw bytes.

However, graph-based detection should not be described as inherently polymorphism-resistant. Control-flow graphs may be altered through opaque predicates, control-flow flattening, function splitting, indirect jumps, and dead-code insertion. Dynamic graphs may also change if malware delays actions or conditions behavior on environmental signals. Robustness must therefore be demonstrated experimentally rather than inferred from the use of graph neural networks alone.

4) Defensive Uses of Large Language Models

Large language models are also being studied as defensive tools for malware analysis, reverse engineering, behavioral summarization, and classification. Surveys of language models in software security identify malware analysis as a growing defensive application [42]. Models can translate technical telemetry into natural-language explanations, summarize code behavior, assist analysts with reverse engineering, or embed behavioral reports for classification.

The defensive potential of language models does not eliminate their reliability limitations. Model outputs may be incorrect, sensitive to prompting, or difficult to reproduce. They may also inherit vulnerabilities to adversarial inputs or manipulation. Defensive language-model systems should therefore operate as components in evidence-based pipelines rather than as unverified autonomous decision-makers.

E. Explainability and Analyst-Centered Detection

Explainable artificial intelligence is relevant to malware detection because security analysts need to understand why a sample was classified as malicious and which features or behaviors influenced the decision. A broad survey of explainable artificial intelligence for malware analysis identifies techniques for feature attribution, local explanation, model interpretation, and analyst support [47].

Explainability is particularly important in environments where false positives carry operational costs. An alert that identifies suspicious application programming interface sequences, control-flow regions, imported functions, or behavioral relationships may be easier to investigate than an unexplained numerical score.

However, explanations may themselves become an attack surface. Mia and Pritom examine adversarial attacks on explanations in cybersecurity applications [48]. A model may retain the same prediction while producing a misleading explanation, or an attacker may manipulate the features emphasized by the explanation system.

A broader survey of adversarial attacks and defenses in explainable artificial intelligence confirms that explanations can be unstable or manipulated [49]. Noppel and Wressnegger further systematize explainable machine learning in adversarial environments, emphasizing that explanations should be evaluated under a defined threat model [50].

Research combining explainability, generative adversarial methods, and mutated-malware detection illustrates efforts to improve both robustness and interpretability [51]. The literature therefore suggests that explainability should not be treated as a cosmetic output layer. It must be tested for faithfulness, robustness, and operational usefulness.

F. Governance Foundations

The governance literature relevant to generative-AI-driven polymorphic malware is broader than the technical malware literature. The National Institute of Standards and Technology Artificial Intelligence Risk Management Framework establishes functions for governing, mapping, measuring, and managing artificial-intelligence risks [19]. Its Generative Artificial Intelligence Profile adapts these principles to risks associated with generative models [20].

ISO/IEC 42001 provides an organizational management-system standard for artificial intelligence [22]. It supports policies, accountability, risk assessment, monitoring, and continuous improvement. The European Union Artificial Intelligence Act introduces legal requirements based on system-risk classifications and organizational responsibilities [21]. These instruments can support governance of dual-use generative systems, but none was designed specifically as a polymorphic-malware control framework.

NIST guidance on managing misuse risk for dual-use foundation models provides a closer connection to malicious capability [23]. It addresses evaluation, safeguards, access, monitoring, and incident handling for models that may support harmful activities. MITRE ATLAS offers a threat-modeling framework for adversarial threats involving artificial-intelligence systems [12].

The Joint Cyber Defense Collaborative playbook adds an ecosystem and incident-collaboration perspective [52]. Threat-intelligence reporting from providers and ENISA contributes evidence about observed misuse and the changing threat landscape [13], [14].

Across these sources, three governance layers emerge: model-provider controls, including misuse evaluation, access management, abuse monitoring, documentation, and escalation; deploying-organization controls, including model inventories, logging, red-teaming, human authorization, isolated testing, and incident response; and ecosystem controls, including common benchmarks, threat sharing, coordinated disclosure, standards, and regulatory accountability.

The literature provides substantial governance scaffolding, but malware-specific implementation and measurement remain limited. Few frameworks define how to test whether a model can generate function-preserving variants, how to measure mutation capability over time, or how to share indicators derived from model-assisted malware incidents. This gap connects the governance literature directly to the technical challenges examined later in the paper.

IV. METHODS

A. Review Design and Research Questions

This study used a structured literature review with systematic search and screening elements. This design was selected because the research problem spans several partially overlapping bodies of literature, including polymorphic and metamorphic malware, adversarial machine learning, generative code models, malware detection, explainable artificial intelligence, evaluation methodology, and AI governance. A fully systematic review was not claimed because the study did not attempt exhaustive retrieval from every bibliographic database, duplicate independent screening by multiple reviewers, or formal meta-analysis. Instead, the method emphasized transparent source selection, explicit inclusion and exclusion criteria, structured evidence extraction, and thematic synthesis.

The review addressed the following main research question:

How is generative AI reshaping the generation and adaptation of polymorphic malware, how well do current detection approaches withstand these variants, and which governance controls are most relevant to the resulting dual-use risk?

Four supporting questions guided source selection and coding:

1. What generative mechanisms produce function-preserving structural or behavioral diversity, and what empirical evidence demonstrates their effectiveness?
2. How robust are static, behavioral, graph-based, and explainable detection approaches against AI-generated or AI-mutated malware?
3. Which datasets, temporal evaluation methods, and benchmarks are required for credible assessment under concept drift and generative mutation?
4. Which provider, organization, and ecosystem-level governance controls can reduce dual-use malware-generation risk, and where do current frameworks remain too general?

The study treated the literature review itself as an analytical process rather than a collection of isolated paper summaries. Sources were compared according to their threat assumptions, transformation mechanisms, malware validity requirements, datasets, detection techniques, evaluation designs, evidence maturity, and governance implications.

B. Search Strategy and Source Identification

Sources were identified through targeted searches of major scholarly, governmental, standards, and professional repositories. Academic sources were drawn primarily from IEEE Xplore, the ACM Digital Library, USENIX, NDSS proceedings, SpringerLink, ScienceDirect, Wiley, arXiv, and related publisher records. Governance and policy sources were retrieved from NIST, EUR-Lex, ISO, ENISA, CISA, NSA, MITRE, and other official institutional repositories. Industry reports were included only when they provided direct evidence of observed misuse, operational threat activity, or defensive practices not yet represented adequately in peer-reviewed literature.

Search terms were organized into four conceptual groups: malware mutation; generative and adversarial methods; detection and evaluation; and governance and misuse.

- Malware mutation: “polymorphic malware,” “metamorphic malware,” “malware obfuscation,” “malware mutation,” and “function-preserving malware transformation.”
- Generative and adversarial methods: “large language model malware,” “generative AI malware,” “LLM code mutation,” “GAN malware evasion,” “reinforcement learning malware,” “AI-generated malware,” and “sandbox evasion.”
- Detection and evaluation: “behavioral malware detection,” “graph neural network malware detection,” “explainable malware detection,” “certified robustness malware,” “concept drift malware,” “malware benchmark,” and “endpoint malware detection.”
- Governance and misuse: “dual-use foundation model misuse,” “AI governance cybersecurity,” “generative AI risk management,” “AI incident sharing,” and “AI malware governance.”

Searches prioritized literature published from 2020 through July 2026 because the most direct research on generative-AI-assisted malware emerged during this period. Earlier sources were retained when they established foundational concepts, datasets, adversarial-machine-learning principles, or historical malware techniques.

The initial evidence pool contained 52 records. One duplicate and misattributed record was removed after its arXiv identifier was found to correspond to a different paper. Ten targeted supplementary sources were then added to strengthen coverage of generative mutation, governance, robustness, runtime adaptation, and evaluation, producing a 61-source working corpus. A subsequent relevance and quality audit removed ten sources that were redundant, peripheral, overly general, or weaker than available alternatives. Four additional sources were introduced to strengthen behavioral detection, endpoint-transfer evaluation, and defensive uses of large language models. The resulting final evidence base contained 55 sources.

Figure 2 summarizes the structured source-identification, verification, audit, and final-inclusion process. The diagram is intended to provide transparent record accounting without implying full PRISMA compliance.

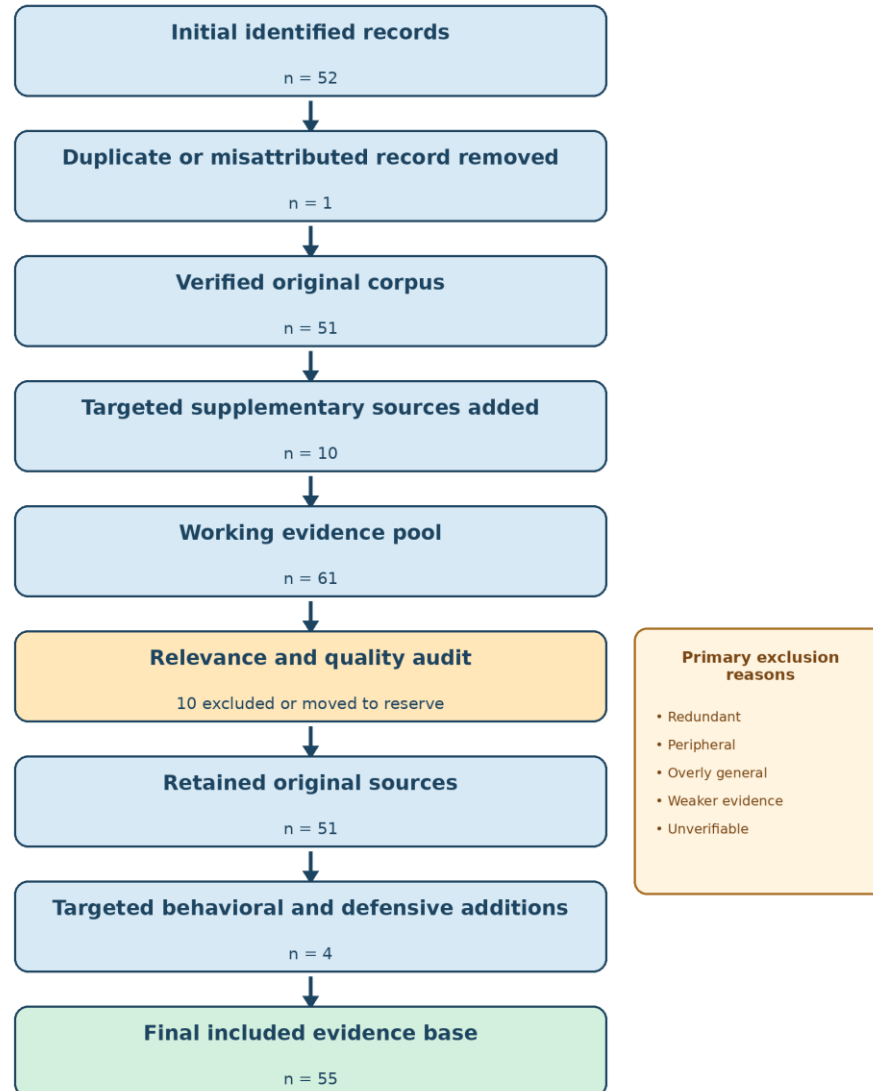


Fig. 2. Structured source-identification, verification, relevance-and-quality audit, and final-inclusion process.

TABLE III. REVIEW PROTOCOL

Element	Review procedure
Review type	Structured literature review with systematic search and screening elements
Primary publication period	2020-July 2026
Historical exceptions	Foundational polymorphism, adversarial-ML, dataset, and evaluation studies
Academic repositories	IEEE, ACM, USENIX, NDSS, Springer, ScienceDirect, Wiley, and arXiv
Governance repositories	NIST, ISO, EUR-Lex, ENISA, CISA, NSA, and MITRE
Initial identified records	52
Duplicate or misattributed records removed	1
Targeted supplementary additions	10
Working evidence pool	61
Sources excluded or moved to reserve	10
Targeted behavioral and defensive additions	4
Final included sources	55
Reconciled evidence profile	37 peer-reviewed journal/conference; 1 published workshop proceedings; 8 current preprints; 8 authoritative governance/standards/framework; 1 industry/provider report
Current preprint share	8 of 55 sources (14.5%)
Evidence extraction	Bibliographic metadata, threat mechanism, detection method, dataset, contribution, limitation, relevance, and taxonomy classification
Synthesis method	Thematic and claim-to-evidence synthesis

C. Inclusion, Exclusion, and Quality Assessment

A source was included when it satisfied at least one of the following criteria:

1. examined traditional polymorphic or metamorphic malware relevant to the review's conceptual baseline;
2. evaluated GAN, reinforcement-learning, or LLM-assisted malware mutation, obfuscation, or evasion;
3. proposed or evaluated a malware-detection technique relevant to structurally or behaviorally diverse variants;
4. addressed datasets, temporal bias, concept drift, robustness, reproducibility, or endpoint deployment;
5. examined explainability or adversarial manipulation of malware-analysis explanations; or
6. provided an authoritative governance, standards, regulatory, or threat-modeling framework applicable to dual-use generative models.

Sources were excluded or moved to reserve status when they were duplicated, misattributed, insufficiently connected to malware mutation, superseded by stronger studies, published through questionable venues, based on unverifiable claims, or too general for the paper's focused scope. Studies addressing phishing, deepfakes, misinformation, prompt injection, or general AI security were excluded unless they contributed directly to malware-generation or dual-use governance analysis.

Each source was assigned to one of five evidence classes: peer-reviewed journal or conference paper; published workshop proceedings; current preprint; authoritative governance, standards, regulatory, or threat-modeling publication; or industry/provider report.

Peer-reviewed studies were prioritized for central technical claims. Following publication-status reconciliation, the final 55-source corpus contains 37 peer-reviewed journal or conference papers, one published workshop-proceedings paper, eight current preprints, eight authoritative governance, standards, regulatory, or threat-modeling sources, and one industry/provider report. The eight current preprints (14.5% of the corpus) were retained because they address recent or underrepresented aspects of the topic; they are treated as emerging or supporting evidence and are not used as the sole basis for broad operational claims. Authoritative government, standards, and regulatory materials are evaluated separately because their role is governance rather than technical performance validation.

Source quality was assessed according to publication status, venue credibility, methodological transparency, malware-validity testing, dataset quality, reproducibility, temporal realism, and relevance to the research questions. Evidence maturity was interpreted in context: convergent peer-reviewed technical findings were weighted more strongly for performance claims; current preprints were treated as emerging evidence; and primary standards, regulations, and government frameworks were treated as authoritative governance evidence rather than grouped with technical preprints.

D. Data Extraction and Evidence Coding

For each included source, the review recorded complete bibliographic metadata; source type and peer-review status; threat or defensive category; malware type or attack surface; AI or optimization technique; detection technique; dataset and evaluation method; principal contribution; principal limitation; practical relevance; related paper section; and taxonomy classification.

The extracted evidence was organized in a 55-source literature matrix. A separate claim-to-evidence map connected 28 proposed claims to primary and supporting sources. Each claim was assigned an evidence-strength label strong, moderate, emerging, methodological, or evidence-boundary and an explicit qualification describing what the evidence did not establish.

This step was important because several studies use similar terms for technically different constructs. For example, a feature-space adversarial attack may cause misclassification without producing a valid executable. A generated code sample may be syntactically correct but fail to compile or preserve malicious functionality. Conversely, a valid executable transformation may preserve behavior while altering only a narrow set of detector features. Coding therefore distinguished among feature-space perturbation, realizable executable transformation, syntactic code mutation, function-preserving semantic mutation, behavioral adaptation, and environment-aware or feedback-driven adaptation.

E. Taxonomy Development and Synthesis

The initial literature matrix grouped studies under threat, detection, and governance categories. Although useful for measuring coverage, that structure did not provide a true analytical taxonomy. The final framework therefore classified offensive techniques using two axes.

The first axis captures mutation and adaptation depth: syntactic or representational mutation; semantic or function-preserving mutation; behavioral or runtime adaptation; and context or environment-aware adaptation. The second axis captures the generative control paradigm: classical or rule-based transformation; GAN, reinforcement-learning, or adversarial optimization; prompted or fine-tuned LLM generation; and feedback-driven or agentic orchestration.

Detection and governance were treated as response overlays. Detection approaches were grouped into static and structural, dynamic and behavioral, semantic and graph-based, explainable, and robustness-oriented defenses. Governance controls were mapped across model-provider, deploying-organization, and ecosystem levels.

Figure 3 operationalizes the review taxonomy by mapping mutation and adaptation depth against the generative control paradigm. The matrix also identifies evidence maturity and positions detection and governance as response overlays rather than additional threat categories.

		Generative control paradigm			
		Rule-based transformation	GAN/RL and adversarial optimization	Prompted or fine-tuned LLM generation	Feedback-driven or agentic orchestration
Mutation and adaptation depth	Context / environment-aware	Predefined VM, timing, or sandbox checks Background	Detector-reward adaptation Moderate	LLM-generated bypass logic Emerging	Closed-loop context-aware orchestration Sparse / gap
	Behavioral / runtime	Traditional anti-analysis routines Background	Behavior-selection policies Moderate	Generated runtime scripts and behaviors Emerging	Multi-stage behavior refinement Sparse
	Semantic / function-preserving	Classical metamorphic rewriting Mature	Executable-preserving RL mutation Moderate	LLM-generated equivalent variants Emerging	Generate-test-repair loops Sparse
	Syntactic / representational	Packing, encryption, and instruction substitution Mature	Feature, byte, and section optimization Established	Source and assembly rewriting Emerging	Iterative surface-level mutation Sparse
Detection overlay:		Static/structural → Dynamic/behavioral → Graph/semantic → Robustness and endpoint evaluation			
Governance overlay:		Model-provider controls → Deploying-organization controls → Ecosystem standards and accountability			

Fig. 3. Two-axis taxonomy of generative-AI-driven polymorphic malware based on mutation depth and generative control paradigm. Detection and governance are represented as response overlays, while cell shading denotes evidence maturity.

The synthesis proceeded thematically rather than chronologically or paper by paper. Studies were compared according to the type of mutation they produced, whether functionality was verified, the detector or environment targeted, and the realism of the evaluation. Areas with strong evidence were separated from emerging capabilities and unresolved research gaps.

F. Methodological Limitations

The method has several limitations. First, the rapid publication cycle of generative-AI security research means that several of the most direct studies remain in preprint form even though only eight of the 55 final sources are current preprints. Second, terminology is inconsistent across malware, adversarial-learning, and generative-AI communities, making automated retrieval and classification imperfect. Third, some studies report success against individual classifiers, antivirus engines, or sandbox environments that may not represent modern enterprise endpoint systems. Fourth, the review did not conduct a statistical meta-analysis because the studies used incompatible datasets, threat models, transformation constraints, and evaluation metrics.

The review also did not independently reproduce malware-generation or evasion experiments. Evidence was assessed through reported methods, validity checks, datasets, and results. No malware samples, proof-of-concept executables, or exploit repositories were accessed as part of the review process.

Finally, bibliographic and automated research tools supported source discovery, metadata organization, and duplicate detection. Final inclusion decisions, claim classifications, evidence qualifications, taxonomy development, and thematic synthesis were subjected to manual academic review. This distinction is important because the final analysis reflects the authors' interpretation of the evidence rather than automated aggregation alone.

V. CHALLENGES

The literature reveals that the challenge posed by generative-AI-driven polymorphic malware is not limited to generating more variants. It also concerns how variants are defined, validated, detected, evaluated, explained, and governed. As illustrated in Fig. 3, the evidence is strongest for traditional and adversarially optimized mutation,

moderate for LLM-driven syntactic and semantic mutation, and considerably thinner for runtime-adaptive and context-aware workflows. This uneven evidence base creates several technical and methodological challenges.

A. Construct Validity and Functional Preservation

The first challenge is determining what should legitimately be classified as polymorphic or metamorphic malware. Studies across malware research, adversarial machine learning, and generative code models often use overlapping terms for technically different phenomena. Table I distinguishes among polymorphism, metamorphism, obfuscation, adversarial malware examples, AI-generated code, and environment-aware adaptation, but many published evaluations do not apply these distinctions consistently.

A feature-space adversarial example may alter imported-function indicators, byte distributions, or classifier features without demonstrating that the changes can be realized in an executable file. Such a study can reveal weaknesses in a classifier's representation, but it does not necessarily demonstrate operational malware mutation. Similarly, source code generated by a language model may appear syntactically plausible while failing to compile, execute, preserve malicious behavior, or operate within the required architecture and runtime environment.

Functional preservation is therefore central to construct validity. A claim of polymorphism should ordinarily demonstrate that the transformed sample remains syntactically valid, compiles or executes successfully, retains the behavior necessary to achieve its intended function, and exhibits meaningful structural or behavioral variation.

Traditional metamorphic-malware studies provide a useful baseline because they emphasize semantic equivalence despite implementation changes [1], [2]. Adversarial-malware research further demonstrates that realizability is a major constraint when moving from classifier feature space to executable problem space [6]. Recent LLM studies improve the evidence base by measuring structural diversity or evaluating code mutation directly [9]-[11], but validation procedures remain inconsistent.

This inconsistency creates a risk of overstating the threat. A large number of structurally different outputs does not necessarily represent a large number of operationally effective malware variants. Conversely, a small structural change may still be strategically important if it consistently bypasses a detector while preserving functionality. Future studies must therefore report not only diversity and evasion rates, but also compilation success, execution success, behavioral equivalence, and transformation provenance.

B. Benchmarks, Temporal Bias, and Concept Drift

A second challenge concerns the suitability of existing malware datasets and evaluation practices. EMBER and SOREL-20M have made important contributions to reproducible static malware research [15], [16]. However, these datasets were not designed specifically to represent LLM-generated, function-preserving, runtime-adaptive, or environment-aware variants.

This limitation does not make legacy datasets obsolete. They remain valuable for baseline comparison, feature research, and large-scale classifier development. The problem arises when performance on such datasets is interpreted as evidence of robustness against newer mutation paradigms that were absent from the data.

Temporal realism is equally important. TESSERACT demonstrated that malware-classification performance can be inflated when training and testing data are divided in ways that do not reflect real deployment conditions [17]. Random splits may allow closely related samples, campaign artifacts, or temporally adjacent variants to appear in both training and testing sets. The resulting model may appear highly accurate while failing when deployed against later malware distributions.

Subsequent work on dataset curation and concept drift has reinforced this concern [53]-[55]. Malware families evolve, benign software ecosystems change, and attacker behavior adapts to defensive controls. Generative systems could accelerate these shifts by increasing the speed with which structurally novel variants are produced. A detector that performs well on historical malware may therefore degrade rapidly even when its original benchmark results remain strong.

Credible evaluation should include chronological train-test separation, malware-family and campaign separation, unseen mutation mechanisms, multiple detector architectures, explicit measurement of drift over time, reproducible transformation records, and evaluation against both legacy and GenAI-specific variants.

The limited availability of GenAI-specific malware datasets is a major obstacle. MalGEN and the AI-generated PowerShell dataset begin to address this gap [32], [33], but both remain recent and specialized. A broadly accepted benchmark covering compiled binaries, scripts, behavioral traces, mutation provenance, functional validation, and endpoint telemetry has not yet emerged.

C. Behavioral Transfer and Environment-Aware Evasion

Behavioral detection is often proposed as a response to polymorphism because malicious objectives usually require observable runtime actions. File creation, registry modification, process injection, credential access, persistence, command execution, and network communication may remain detectable even when code structure changes.

However, behavioral detection faces its own transferability problem. Models trained in sandboxes may learn patterns specific to the sandbox environment rather than patterns that generalize to production endpoints. Endpoint systems contain legitimate background activity, organization-specific software, user interaction, variable permissions, security agents, and hardware differences. These factors can change both malware behavior and the telemetry available to the detector.

Research examining the transfer of behavior-based malware detection to endpoints identifies a measurable gap between sandbox-trained models and deployment conditions [18]. This finding is particularly important for generative-AI-driven malware because a model may generate logic that intentionally reacts to environmental cues.

Sandbox-aware malware can delay execution, require specific user activity, inspect hardware or system artifacts, detect virtualization, check process and service configurations, alter behavior based on network access, or execute only when a selected condition is satisfied.

Recent work on LLM-generated sandbox-bypass rules suggests that generative models may assist in constructing such logic [31]. However, the evidence remains limited and should not be interpreted as proof that autonomous context-aware malware is already widespread.

Behavioral detection also introduces operational costs. Dynamic execution requires time, isolation, instrumentation, and secure handling. Some malicious branches may remain unobserved because the analysis window is too short or the required trigger is absent. High-volume environments may therefore struggle to execute every suspicious artifact deeply enough to expose its complete behavior.

The challenge is not choosing between static and behavioral analysis. It is combining rapid static screening with selectively deeper behavioral and endpoint analysis, while ensuring that the resulting models are tested across varied environments.

D. Robustness Guarantees and Adaptive Attacks

Robustness-oriented malware detection attempts to establish how a classifier behaves under adversarial modification. Certified approaches are especially valuable because they define a bounded set of perturbations within which a prediction is guaranteed to remain stable [34], [35].

The principal challenge is that certification applies only to the specified threat model. A detector certified against appended bytes may still be vulnerable to control-flow rewriting, semantic mutation, behavioral changes, or context-sensitive execution. Similarly, robustness against one executable format or feature representation does not establish robustness across scripts, documents, memory-resident malware, or multistage attacks.

Accuracy and coverage trade-offs also arise. Stronger guarantees may reduce the number of samples for which a model can provide a certificate, or they may lower conventional classification performance. Emerging robustness-by-design approaches attempt to improve this balance [43], but the field remains experimental.

Adaptive attackers create a further complication. Once a detector, explanation method, or transformation constraint is known, the attacker may optimize around it. A robust evaluation should therefore include attacks tailored to the evaluated model, attacks transferred from surrogate models, previously unseen transformation types, repeated or composed transformations, semantic validity checks, and realistic query and feedback limitations.

This is especially relevant to LLM-driven mutation because language models may generate transformations outside the narrow perturbation sets used in earlier adversarial-malware research. Robustness must therefore be assessed against a changing space of transformations rather than a fixed library alone.

E. Trustworthy Explainable Artificial Intelligence

Explainable artificial intelligence is intended to help analysts understand why a sample or behavior was classified as malicious. In malware analysis, explanations may identify suspicious imports, application programming interface calls, graph regions, strings, byte ranges, or behavioral relationships [47].

The challenge is that an explanation can appear plausible without being faithful to the model's actual reasoning. It may also be unstable: small changes to the input can produce different explanations even when the prediction remains unchanged. In adversarial settings, an attacker may deliberately manipulate the explanation to hide malicious features, emphasize benign-looking evidence, or misdirect analyst attention.

Research on adversarial attacks against cybersecurity explanations demonstrates that the explanation layer can become a separate attack surface [48]. Broader studies of adversarial explainability show that interpretability methods require their own threat models and robustness evaluation [49], [50].

This creates a difficult operational problem. Analysts may trust an explanation precisely because it appears understandable, even when it is incomplete or manipulated. Explainable malware detection should therefore be evaluated for faithfulness to the underlying model, stability under benign input variation, resilience to adversarial manipulation, consistency with observable behavior, usefulness during investigation, and compatibility with human decision-making.

Explainability should support analyst judgment rather than replace it. A model explanation is evidence to be assessed, not an authoritative account of malicious intent.

F. Governance Specificity, Enforcement, and Measurement

The final challenge is translating broad AI governance principles into malware-specific controls. NIST AI RMF, the NIST Generative AI Profile, ISO/IEC 42001, the European Union Artificial Intelligence Act, MITRE ATLAS, and NIST misuse-risk guidance provide valuable structures for risk identification, accountability, monitoring, and incident response [12], [19]-[23].

However, these frameworks generally operate at a high level. They do not define a standardized test for determining whether a model can generate function-preserving malware variants, nor do they establish common thresholds for acceptable code-mutation capability, abuse-monitoring effectiveness, or sandbox-evasion risk.

Governance effectiveness is also difficult to measure. A model provider may state that it performs red-teaming or abuse monitoring, but external stakeholders may have limited visibility into which malware-generation scenarios were tested, how frequently evaluations are repeated, how safeguards are updated, what constitutes a reportable incident, whether restricted capabilities can be reconstructed through fine-tuning or tool use, and how findings are shared across providers and defenders.

Deploying organizations face similar difficulties. Policies, model inventories, access controls, and logging are useful only when they are implemented, tested, and assigned to accountable owners. Smaller organizations may lack the personnel or infrastructure to conduct specialized model-risk assessments.

The governance problem therefore extends across model providers, deploying organizations, and the wider ecosystem. Model providers must evaluate misuse capabilities, control access, monitor abuse, and document safeguards. Deploying organizations must restrict sensitive use, isolate testing, maintain logs, and integrate AI incidents into existing response processes. The wider ecosystem must develop common benchmarks, reporting practices, threat-sharing mechanisms, and accountability standards.

The Joint Cyber Defense Collaborative playbook and MITRE ATLAS provide useful foundations for coordination and threat modeling [12], [52]. Nevertheless, enforcement, comparability, and measurement remain immature. Governance frameworks can identify responsibilities, but their practical effectiveness against generative-AI-driven malware will depend on whether those responsibilities are converted into observable and testable controls.

G. Challenge Synthesis

The challenges identified in this section are closely connected. Weak definitions undermine benchmark validity. Weak benchmarks make robustness claims difficult to interpret. Poor endpoint transfer limits behavioral detection. Manipulable explanations can undermine analyst confidence. Generic governance frameworks may fail to measure whether these technical risks are being addressed.

The review therefore suggests that no single detector, dataset, or policy framework is sufficient. A credible response requires clearer construct definitions, functionally validated mutation benchmarks, temporally realistic testing, representation-diverse detection, adversarially evaluated explanations, and measurable governance controls. These findings provide the basis for the recommendations developed in Section VI.

VI. RECOMMENDATIONS

Figure 4 presents the integrated technical and governance response developed from the review findings, while Table IV maps each principal challenge to its recommended action, responsible stakeholders, intended outcome, and implementation limitation.

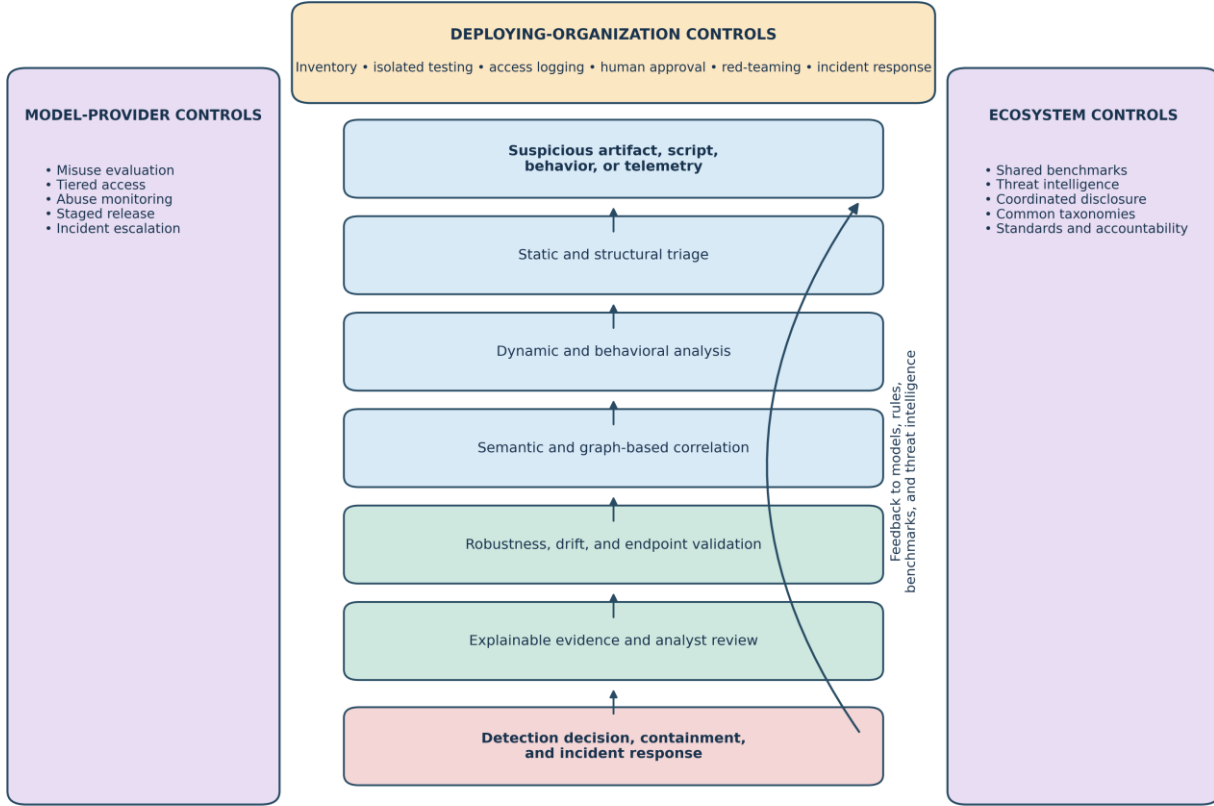


Fig. 4. Layered technical and governance response to generative-AI-driven malware mutation, integrating representation-diverse detection, endpoint validation, analyst review, and multilevel risk controls.

A. Establish Generative-AI-Specific Evaluation Benchmarks

The most immediate research priority is the development of benchmarks designed specifically for generative-AI-driven malware mutation. Existing datasets such as EMBER and SOREL-20M remain valuable for static malware research, but they do not adequately represent large populations of AI-generated, function-preserving, behaviorally adaptive, or environment-aware variants [15], [16].

A credible GenAI-specific benchmark should record more than the final malware sample. It should preserve the complete transformation history, including the original sample or behavior specification; the model and model version used; the prompting, fine-tuning, or optimization process; each transformation applied; compilation and execution outcomes; functional-equivalence tests; malware-family and campaign metadata; detector-query information; and the final evasion or detection outcome.

This provenance is essential because it allows researchers to distinguish genuine function-preserving mutation from invalid code, superficial rewriting, or feature-space manipulation. A sample should not be classified as a successful polymorphic variant merely because its structure changed or one detector misclassified it.

Evaluation datasets should also include several malware representations. Native executables, scripts, behavioral traces, control-flow graphs, application programming interface sequences, and endpoint telemetry each expose different aspects of malicious activity. MalGEN and the AI-generated PowerShell framework provide useful starting points, but their scope remains limited [32], [33].

Benchmark design should apply chronological splits and separate related samples by malware family, campaign, and mutation lineage. TESSERACT and subsequent concept-drift studies show that random data division can inflate apparent performance and conceal deployment weaknesses [17], [53]-[55]. New benchmarks should therefore evaluate whether detectors generalize to future variants rather than merely recognizing closely related samples.

Because malware experimentation creates safety risks, benchmark development should occur in controlled environments with clear access restrictions, responsible release practices, and nonweaponized representations where possible. Public datasets may expose extracted features, execution traces, or transformed representations without distributing operational malware binaries.

B. Deploy Layered, Representation-Diverse Detection

No single detection approach is likely to remain reliable across every mutation mechanism shown in Fig. 3. Static, behavioral, semantic, graph-based, and robustness-oriented approaches should therefore be combined as complementary layers.

A practical pipeline may begin with rapid static and structural analysis. Hashes, signatures, headers, imports, strings, byte distributions, and learned static features remain effective for known threats and high-volume triage. Their efficiency makes them unsuitable for abandonment, even though they are vulnerable to representation changes.

Samples that remain uncertain or exhibit suspicious structural characteristics should proceed to dynamic and behavioral analysis. Runtime evidence - such as process creation, application programming interface calls, persistence activity, file modification, memory behavior, and network communication - can reveal malicious intent that is not visible through static representation alone [37], [44], [45].

Graph and semantic models can add a further layer by analyzing control flow, call relationships, entity interactions, and behavior dependencies [38]-[41]. Such models may identify relationships that survive surface-level rewriting. However, they should not be assumed to be inherently resistant to adversarial modification. Their contribution is strongest when combined with other evidence rather than treated as a universal replacement.

Ensemble approaches can improve resilience by requiring an attacker to manipulate several representations simultaneously. A variant that bypasses a static classifier may still expose suspicious runtime behavior, while a sample that suppresses sandbox activity may exhibit unusual control-flow or endpoint telemetry. Detection systems should therefore aggregate heterogeneous evidence and calibrate their confidence rather than relying on a single opaque score.

Large language models may assist with behavioral summarization, reverse-engineering support, alert correlation, or interpretation of sandbox reports [42], [45], [46]. Their role should remain advisory. Decisions involving containment, escalation, or malware-family classification should be grounded in observable technical evidence and validated by analysts.

C. Require Robustness, Drift, and Endpoint-Transfer Testing

Malware-detection studies should report robustness using explicit and reproducible threat models. Statements such as “robust against adversarial malware” are too broad unless the permitted transformations, attacker knowledge, feedback access, and functional constraints are clearly defined.

Robustness evaluation should include white-box and black-box attacks, surrogate-model transfer, unseen mutation mechanisms, repeated and composed transformations, limited-query conditions, functionality-preserving constraints, temporal drift, detector-version changes, and deployment in endpoint-relevant environments.

Certified robustness methods provide valuable guarantees, but those guarantees apply only to the perturbation classes included in the certification process [34], [35]. Researchers and vendors should report the exact boundary of each guarantee and avoid generalizing it to semantic rewriting, runtime adaptation, or context-aware execution.

Empirical robustness should also be assessed under adaptive attack. A detector evaluated only against a fixed library of transformations may appear effective because the attacker has not optimized against its specific architecture. Evaluation should assume that capable adversaries may learn from feedback and alter their strategies accordingly.

Concept drift should be treated as an operational condition rather than an occasional research variable. Organizations should monitor changes in feature distributions, malware-family prevalence, false-positive rates, confidence calibration, and detection latency. Models should be re-evaluated periodically using recent samples and chronological holdout sets.

Endpoint-transfer testing is equally important. Behavioral systems trained in sandboxes should be evaluated on representative endpoint telemetry before operational deployment. Shadow-mode testing, staged rollout, and analyst feedback can help identify performance differences without immediately affecting production decisions. Organizations should avoid assuming that sandbox accuracy transfers directly to live environments.

D. Harden Explainability and Analyst Workflows

Explainable artificial intelligence should be treated as part of the security boundary. If analysts rely on explanations to prioritize investigations or justify containment decisions, attackers may benefit from manipulating those explanations even when they cannot change the final prediction.

Explanation systems should be tested for faithfulness to the underlying model, stability across minor benign variations, consistency with observable malware behavior, resilience against adversarial manipulation, usefulness to analysts, and repeatability across model versions.

Adversarial-XAI research demonstrates that explanations can be altered or misdirected [48]-[50]. Vendors should therefore evaluate whether a sample can preserve its malicious classification while changing the features emphasized in the explanation. They should also test whether benign-looking features can be amplified to distract analysts from more important evidence.

Explanations should cite the technical basis for their conclusions. A behavioral explanation should link to observed application programming interface calls, processes, files, network events, or memory indicators. A graph-based explanation should identify the relevant nodes, relationships, and paths. Natural-language summaries generated by large language models should remain traceable to the underlying evidence.

Human review remains essential. Explainability should reduce analyst workload and support decision quality, not transform probabilistic model output into an apparently authoritative narrative. Organizations should maintain escalation procedures for cases where the model prediction, explanation, and observed telemetry do not agree.

E. Operationalize Layered Governance

1) Model-Provider Controls

Providers of general-purpose and code-capable models should evaluate misuse capabilities before and after release. Testing should include malware-related scenarios appropriate to the model's capability level, such as code mutation, obfuscation, compilation repair, sandbox-evasion assistance, and iterative generate-test-revise workflows.

Relevant controls include tiered access to high-risk capabilities, misuse-oriented red-teaming, abuse monitoring, rate and query controls, staged release, model and system documentation, incident escalation, and periodic reassessment after model updates.

NIST AI RMF, the Generative AI Profile, ISO/IEC 42001, and dual-use foundation-model guidance provide structures for these activities [19], [20], [22], [23]. The challenge is to convert high-level principles into measurable controls. Providers should document which misuse scenarios were tested, what limitations were found, and how safeguards are monitored over time.

2) Deploying-Organization Controls

Organizations using generative models for software development, malware analysis, cybersecurity research, or automation should maintain an inventory of models, connected tools, authorized users, and approved use cases.

Sensitive capabilities should be subject to role-based access, isolated execution, prompt and tool-use logging, approval requirements, output scanning, secure retention, incident-response integration, and periodic control testing.

Cybersecurity teams should explicitly include AI-assisted malware misuse within threat models and incident-response plans. MITRE ATLAS can support identification of AI-related attack techniques, while existing malware-response procedures can be extended to include model, prompt, tool, and output provenance [12].

3) Ecosystem Controls

The wider ecosystem requires common benchmarks, shared incident terminology, threat-intelligence exchange, and coordinated disclosure. Providers and defenders may otherwise identify similar misuse patterns independently without an effective mechanism for comparison.

The JCDC playbook and existing threat-sharing structures provide useful foundations [52]. Future collaboration should support common definitions for AI-assisted malware incidents, sharing of nonweaponized indicators, standardized reporting of model-assisted mutation, coordinated disclosure between researchers and providers, benchmark-development partnerships, and independent evaluation of misuse controls.

Regulatory and standards bodies should avoid prescribing narrow technical safeguards that may become obsolete quickly. They should instead require measurable risk-management processes, documented evaluations, accountable ownership, incident reporting, and evidence of continual improvement.

The European Union Artificial Intelligence Act introduces legally enforceable obligations for certain AI systems, while NIST and ISO frameworks support broader organizational risk management [19]-[23]. Their effectiveness against generative-AI-driven malware will depend on implementation, oversight, and the availability of reliable technical evaluation methods.

TABLE IV. EVIDENCE-BASED CHALLENGE-TO-RECOMMENDATION MAPPING

Challenge	Evidence-based recommendation	Primary stakeholders	Intended outcome	Principal limitation
Ambiguous definitions and inconsistent validity testing	Require compilation, execution, behavioral-preservation, and transformation-provenance reporting	Researchers, reviewers, and benchmark maintainers	Comparable evidence of genuine polymorphism	Validation may be costly or unsafe
Legacy datasets and temporal bias	Develop GenAI-specific datasets using chronological splits and reproducible mutation records	Researchers, dataset maintainers, and vendors	More realistic performance estimates	Rapid dataset ageing
Sandbox-to-endpoint transfer gap	Evaluate behavioral models using endpoint-relevant telemetry and diverse execution environments	EDR vendors, SOC teams, and evaluators	Improved deployment validity	Access to representative endpoint data
Restricted robustness guarantees	Report explicit perturbation models and test adaptive, transferable, and previously unseen attacks	Model developers and independent evaluators	More defensible robustness claims	Formal guarantees remain threat-model-specific
Manipulable explanations	Adversarially test explanation systems and preserve human review	Security vendors and analysts	More trustworthy analyst support	Explainability does not ensure causal correctness
Generic governance frameworks	Translate AI risk frameworks into measurable provider, organization, and ecosystem-level controls	Model providers, organizations, regulators, and standards bodies	Accountable and testable misuse-risk management	Enforcement and measurement remain immature

F. Recommendation Synthesis

The recommendations converge on a common principle: generative-AI-driven malware should not be addressed through a single new detector or policy. The threat spans code generation, adversarial optimization, software validity, behavioral execution, endpoint deployment, analyst interpretation, and model governance.

An effective response therefore requires benchmarks that verify functional malware mutation; layered detection across multiple representations; temporal, adaptive, and endpoint-oriented evaluation; trustworthy explanation and human review; and measurable governance across providers, organizations, and the cybersecurity ecosystem.

These measures do not eliminate the possibility of generative-AI-assisted malware. They reduce uncertainty, improve the credibility of defensive claims, and make it more difficult for attackers to exploit gaps between research benchmarks, deployed systems, and governance controls.

VII. CONCLUSION

Generative artificial intelligence is reshaping malware development by reducing the effort required to generate, modify, and test structurally diverse code. The reviewed evidence indicates that large language models, generative adversarial networks, and reinforcement-learning systems can support code rewriting, obfuscation, detector-guided evasion, and increasingly adaptive malware-development workflows. However, the literature does not yet establish that generative-AI-driven polymorphic malware consistently defeats modern production endpoint detection and response systems at scale. Much of the most direct evidence remains experimental, narrowly scoped, or preprint-based.

A central finding of this review is that technical distinctions matter. Feature-space adversarial examples, conventional obfuscation, syntactically varied code, function-preserving executable mutation, behavioral adaptation, and environment-aware malware are related but not equivalent. Claims of polymorphism should therefore be supported by evidence of executable validity, preserved malicious functionality, meaningful variation, and reproducible transformation provenance.

Current malware defenses remain useful but incomplete. Static and signature-based techniques provide scalable initial screening, while behavioral, semantic, graph-based, and explainable approaches offer additional resilience against surface-level variation. None of these methods is inherently robust to all mutation strategies. Their effectiveness depends on layered deployment, realistic temporal evaluation, adaptive attack testing, endpoint-relevant telemetry, and continuous monitoring for concept drift.

The review also finds that existing AI governance frameworks provide valuable foundations for accountability, risk assessment, monitoring, and incident response, but remain largely generic. Managing generative-AI-assisted malware requires more specific controls across model providers, deploying organizations, and the cybersecurity ecosystem. These controls should include misuse-capability evaluation, access restrictions, isolated testing, audit logging, responsible disclosure, common benchmarks, and coordinated threat sharing.

Overall, generative AI should be understood as an amplifier of malware polymorphism rather than a complete replacement for established mutation techniques. Its most significant effect may be the acceleration and scaling of iterative malware generation. Future research should prioritize functionally validated GenAI-specific datasets, end-to-end endpoint evaluation, runtime and environment-aware mutation studies, adversarially robust explanation systems, and measurable governance controls. Such work is necessary to distinguish plausible future risk from demonstrated operational capability and to ensure that defensive innovation keeps pace with increasingly automated malware development.

REFERENCES

- [1] K. Brezinski and K. Ferens, "Metamorphic malware and obfuscation: A survey of techniques, variants, and generation kits," *Security Commun. Netw.*, vol. 2023, Art. no. 8227751, 2023, doi: 10.1155/2023/8227751.
- [2] H. Xu, Y. Zhou, J. Ming, and M. Lyu, "Layered obfuscation: A taxonomy of software obfuscation techniques for layered security," *Cybersecurity*, vol. 3, Art. no. 9, Apr. 2020, doi: 10.1186/s42400-020-00049-3.
- [3] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. ICLR*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6572>
- [4] B. Biggio and F. Roli, "Wild patterns: Ten years after the rise of adversarial machine learning," *Pattern Recogn.*, vol. 84, pp. 317-331, Dec. 2018, doi: 10.1016/j.patcog.2018.07.023.
- [5] C. Catalano, A. Chezzi, M. Angelelli, and F. Tommasi, "Deceiving AI-based malware detection through polymorphic attacks," *Comput. Ind.*, vol. 143, Art. no. 103751, Dec. 2022, doi: 10.1016/j.compind.2022.103751.
- [6] L. Demetrio, S. E. Coull, B. Biggio, G. Lagorio, A. Armando, and F. Roli, "Adversarial EXEmpleS: A survey and experimental evaluation of practical attacks on machine learning for Windows malware detection," *ACM Trans. Privacy Secur.*, vol. 24, no. 4, Art. no. 26, Aug. 2021, doi: 10.1145/3473039.
- [7] S. Mohseni, S. Mohammadi, D. Tilwani, Y. Saxena, G. K. Ndawula, S. Vema, E. Raff, and M. Gaur, "Can LLMs obfuscate code? A systematic analysis of large language models into assembly code obfuscation," *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 23, pp. 24893-24901, 2025, doi: 10.1609/aaai.v39i23.34672.
- [8] Y. M. Pa Pa, S. Tanizaki, T. Kou, M. van Eeten, K. Yoshioka, and T. Matsumoto, "An attacker's dream? Exploring the capabilities of ChatGPT for developing malware," in *Proc. 16th Cyber Security Experimentation and Test Workshop (CSET)*, 2023, pp. 10-18, doi: 10.1145/3607505.3607513.

- [9] M. Setak and P. Madani, "Fine-tuning LLMs for code mutation: A new era of cyber threats," in Proc. 2024 IEEE 6th Int. Conf. Trust, Privacy Security Intell. Syst. Appl. (TPS-ISA), 2024, pp. 313-321, doi: 10.1109/TPS-ISA62245.2024.00043.
- [10] G. Horte and J. Tapiador, "The infinite mutation engine? Measuring polymorphism in LLM-generated offensive code," preprint, arXiv:2605.03619 [cs.CR], May 2026, doi: 10.48550/arXiv.2605.03619.
- [11] P. Madani, "Metamorphic malware evolution: The potential and peril of large language models," in Proc. 2023 5th IEEE Int. Conf. Trust, Privacy Security Intell. Syst. Appl. (TPS-ISA), Atlanta, GA, USA, 2023, pp. 74-81, doi: 10.1109/TPS-ISA58951.2023.00019.
- [12] MITRE Corporation, "MITRE ATLAS: Adversarial threat landscape for artificial intelligence systems," MITRE, 2022. [Online]. Available: <https://atlas.mitre.org/>
- [13] European Union Agency for Cybersecurity, ENISA Threat Landscape 2025. Athens, Greece: ENISA, Oct. 2025, ISBN 978-92-9204-723-8, doi: 10.2824/1946374. [Online]. Available: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025>
- [14] OpenAI, "Disrupting malicious uses of AI by state-affiliated threat actors," OpenAI, Feb. 2024. [Online]. Available: <https://openai.com/index/disrupting-malicious-ai-uses/>
- [15] H. S. Anderson and B. F. Roth, "EMBER: An open dataset for training static PE malware machine learning models," preprint, arXiv:1804.04637 [cs.CR], Apr. 2018.
- [16] R. Harang and E. M. Rudd, "SOREL-20M: A large scale benchmark dataset for malicious PE detection," in Proc. Conf. Applied Machine Learning for Information Security (CAMLIS), 2021, pp. 13-21. [Online]. Available: <https://ceur-ws.org/Vol-3095/paper2.pdf>.
- [17] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in Proc. 28th USENIX Security Symp., 2019, pp. 729-746. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury>
- [18] Y. Kaya, Y. Chen, S. Saha, F. Pierazzi, L. Cavallaro, D. Wagner, and T. Dumitras, "Demystifying behavior-based malware detection at endpoints," arXiv:2405.06124 [cs.CR], May 2024. [Online]. Available: <https://arxiv.org/abs/2405.06124>
- [19] E. Tabassi et al., "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, Nat. Inst. Standards Technol., Jan. 2023, doi: 10.6028/NIST.AI.100-1. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
- [20] Nat. Inst. Standards Technol., "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST AI 600-1, 2024, doi: 10.6028/NIST.AI.600-1. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>
- [21] European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," Off. J. Eur. Union, L, 2024/1689, Jul. 2024. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [22] International Organization for Standardization, "ISO/IEC 42001:2023: Information technology - Artificial intelligence - Management system," ISO, Geneva, Switzerland, Dec. 2023. [Online]. Available: <https://www.iso.org/standard/81230.html>
- [23] U.S. AI Safety Institute, Managing Misuse Risk for Dual-Use Foundation Models, NIST AI 800-1 2pd, 2nd Public Draft. Gaithersburg, MD, USA: Nat. Inst. Standards Technol., Jan. 2025, doi: 10.6028/NIST.AI.800-1.2pd. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.800-1.2pd.pdf>
- [24] W. Hu and Y. Tan, "Generating adversarial malware examples for black-box attacks based on GAN," in Data Mining and Big Data (DMBD 2022), Commun. Comput. Inf. Sci., vol. 1745, pp. 409-423, 2022, doi: 10.1007/978-981-19-8991-9_29.
- [25] Phan The Duy, Duc Luong Tran, Hoang Quoc An Nguyen, Nguyen Huu Quyen, Nghi Hoang Khoa, and Pham Van-Hau, "Leveraging reinforcement learning and generative adversarial networks to craft mutants of Windows malware against black-box malware detectors," in Proc. 11th Int. Symp. Information and Communication Technology (SoICT), 2022, doi: 10.1145/3568562.3568636.
- [26] R. Labaca-Castro, S. Franz, and G. Dreo Rodosek, "AIMED-RL: Exploring adversarial malware examples with reinforcement learning," in Proc. Eur. Conf. Mach. Learn. Principles Pract. Knowl. Discovery Databases (ECML PKDD), 2021, pp. 37-52, doi: 10.1007/978-3-030-86514-6_3.
- [27] M. Rigaki and S. Garcia, "The power of MEME: Adversarial malware creation with model-based reinforcement learning," in Proc. Eur. Symp. Research Comput. Security (ESORICS), 2023, pp. 44-64, doi: 10.1007/978-3-031-51482-1_3.
- [28] D. Comney et al., "EGAN: Evolutional GAN for ransomware evasion," in Proc. 48th IEEE Conf. Local Comput. Netw. (LCN), 2023, doi: 10.1109/LCN58197.2023.10223320.
- [29] D. Gibert, J. Planes, Q. Le, and G. Zizzo, "A wolf in sheep's clothing: Query-free evasion attacks against machine learning-based malware detectors with generative adversarial networks," in Proc. IEEE Eur. Symp. Security Privacy Workshops (EuroS&PW), 2023, doi: 10.1109/EuroSPW59978.2023.00052.
- [30] B. Etter, J. L. Hu, M. Ebrahimi, W. Li, X. Li, and H. Chen, "Evading deep learning-based malware detectors via obfuscation: A deep reinforcement learning approach," in Proc. 2023 IEEE Int. Conf. Data Mining (ICDM), 2023, pp. 101-109, doi: 10.1109/ICDM58522.2023.00019.
- [31] Z. Sui, L. Noureddine, M. E. Khatun, S. Bello, J. Woodring, and A. I. Ali-Gombe, "A large language model approach to generating bypass rules for malware evasion in analysis sandbox," preprint, arXiv:2605.21821 [cs.CR], May 2026. [Online]. Available: <https://arxiv.org/abs/2605.21821>
- [32] B. Saha and S. K. Shukla, "MalGEN: A testbed for modeling and evaluating malware behaviors," arXiv:2506.07586 [cs.CR], Jun. 2025, rev. Apr. 2026. [Online]. Available: <https://arxiv.org/abs/2506.07586>
- [33] L. Pianese, V. Orbinato, P. Liguori, and R. Natella, "AI-generated PowerShell malware: An experimental framework and dataset," preprint, arXiv:2606.30819 [cs.CR], Jun. 2026. [Online]. Available: <https://arxiv.org/abs/2606.30819>
- [34] S. Saha, W. Wang, Y. Kaya, S. Feizi, and T. Dumitras, "DRSM: De-randomized smoothing on malware classifier providing certified robustness," in Proc. Int. Conf. Learn. Representations (ICLR), 2024. [Online]. Available:

https://proceedings.iclr.cc/paper_files/paper/2024/hash/d09ef5264966e17adffd3157265c9946-Abstract-Conference.html.

- [35] D. Gibert, G. Zizzo, and Q. Le, "Certified robustness of static deep learning-based malware detectors against patch and append attacks," in Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec'23), 2023, doi: 10.1145/3605764.3623914.
- [36] M. G. Gaber, M. Ahmed, and H. Janicke, "Malware detection with artificial intelligence: A systematic literature review," ACM Comput. Surv., vol. 56, no. 6, Art. no. 148, pp. 1-33, 2024, doi: 10.1145/3638552.
- [37] C. Li, Z. Cheng, H. Zhu, L. Wang, Q. Lv, Y. Wang, N. Li, and D. Sun, "DMalNet: Dynamic malware analysis based on API feature engineering and graph learning," Comput. Secur., vol. 122, Art. no. 102872, Nov. 2022, doi: 10.1016/j.cose.2022.102872.
- [38] J. Liu, Y. Zhao, Y. Feng, Y. Hu, and X. Ma, "SeMalBERT: Semantic-based malware detection with bidirectional encoder representations from transformers," J. Inf. Secur. Appl., vol. 80, Art. no. 103690, Feb. 2024, doi: 10.1016/j.jisa.2023.103690.
- [39] S.-J. Bu and S.-B. Cho, "Triplet-trained graph transformer with control flow graph for few-shot malware classification," Inf. Sci., vol. 649, Art. no. 119598, Nov. 2023, doi: 10.1016/j.ins.2023.119598.
- [40] X. Ling, L. Wu, W. Deng, Z. Qu, J. Zhang, S. Zhang, T. Ma, B. Wang, C. Wu, and S. Ji, "MalGraph: Hierarchical graph neural networks for robust Windows malware detection," in Proc. IEEE INFOCOM, 2022, pp. 1998-2007, doi: 10.1109/INFOCOM48880.2022.9796786.
- [41] J. Yu, H. Wang, Y. Zhang, W. Zhang, L. Qin, L. Lai, and B. Yang, "MGDN: A graph of graphs neural network for malware detection," IEEE Trans. Knowl. Data Eng., vol. 38, no. 7, pp. 4542-4555, Jul. 2026, doi: 10.1109/TKDE.2026.3689176.
- [42] H. Jelodar, S. Bai, P. Hamed, H. Mohammadian, R. Razavi-Far, and A. Ghorbani, "Large language model (LLM) for software security: Code analysis, malware analysis, reverse engineering," J. Inf. Secur. Appl., vol. 98, Art. no. 104390, May 2026, doi: 10.1016/j.jisa.2026.104390.
- [43] P.-F. Gimenez, S. Sivaprasad, and M. Fritz, "Certifiably robust malware detectors by design," in ICT Systems Security and Privacy Protection, IFIP SEC 2025, IFIP AICT, vol. 746, pp. 125-139, 2025, doi: 10.1007/978-3-031-92886-4_9.
- [44] L. Mauri and E. Damiani, "Hardening behavioral classifiers against polymorphic malware: An ensemble approach based on minority report," Inf. Sci., vol. 689, Art. no. 121499, Jan. 2025, doi: 10.1016/j.ins.2024.121499.
- [45] W. S. Perera and H. Jiang, "BEACON: Behavioral malware classification with large language model embeddings and deep learning," in Proc. 24th Int. Conf. Mach. Learn. Appl. (ICMLA), 2025, pp. 176-183, doi: 10.1109/ICMLA66185.2025.00029.
- [46] R. Saul, J. Jiang, E. Chia, and D. Wagner, "Trident: Improving malware detection with LLMs and behavioral features," arXiv:2605.00297 [cs.CR], Apr. 2026. [Online]. Available: <https://arxiv.org/abs/2605.00297>
- [47] H. Manthena, S. Shajarian, J. Kimmell, M. Abdelsalam, S. Khorsandroo, and M. Gupta, "Explainable artificial intelligence (XAI) for malware analysis: A survey of techniques, applications, and open challenges," IEEE Access, vol. 13, pp. 61611-61640, 2025, doi: 10.1109/ACCESS.2025.3555926.
- [48] M. Mia and M. M. A. Pritom, "Explainable but vulnerable: Adversarial attacks on XAI explanation in cybersecurity applications," in Proc. 2025 IEEE 7th Int. Conf. Trust, Privacy Security Intell. Syst. Appl. (TPS-ISA), Pittsburgh, PA, USA, 2025, pp. 101-110, doi: 10.1109/TPS-ISA67132.2025.00020.
- [49] H. Baniecki and P. Biecek, "Adversarial attacks and defenses in explainable artificial intelligence: A survey," Inf. Fusion, vol. 107, Art. no. 102303, 2024, doi: 10.1016/j.inffus.2024.102303.
- [50] M. Noppel and C. Wressneger, "SoK: Explainable machine learning in adversarial environments," in Proc. 2024 IEEE Symp. Security and Privacy (SP), 2024, pp. 2441-2459, doi: 10.1109/SP54263.2024.00021.
- [51] Truong Thi Hoang Hao, Nguyen Thai Duong, Nghi Hoang Khoa, Van-Hau Pham, and Phan The Duy, "A trustworthy and robust model for mutated malware detection via explainable AI and GANs," in Proc. 2024 Int. Conf. Advanced Technologies for Communications (ATC), 2024, pp. 945-950, doi: 10.1109/ATC63255.2024.10908265.
- [52] Cybersecurity and Infrastructure Security Agency, "JCDC AI Cybersecurity Collaboration Playbook," Joint Cyber Defense Collaborative, Jan. 14, 2025. [Online]. Available: <https://www.cisa.gov/resources-tools/resources/jcdc-ai-cybersecurity-collaboration-playbook>.
- [53] T. Chow, M. D'Onghia, L. Linhardt, Z. Kan, D. Arp, L. Cavallaro, and F. Pierazzi, "Beyond the TESSERACT: Trustworthy dataset curation for sound evaluations of Android malware classifiers," in Proc. 4th IEEE Conf. Secure and Trustworthy Machine Learning (SaTML), 2026, in press. [Online]. Available: <https://arxiv.org/abs/2506.23814>.
- [54] W. Maillet and B. Marais, "Optimized deep learning models for malware detection under concept drift," preprint, arXiv:2308.10821 [cs.CR], Aug. 2023.
- [55] A. S. Li, A. Iyengar, A. Kundu, and E. Bertino, "Revisiting concept drift in Windows malware detection: Adaptation to real drifted malware with minimal samples," in Proc. Netw. Distrib. Syst. Security Symp. (NDSS), 2025. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/revisiting-concept-drift-in-windows-malware-detection-adaptation-to-real-drifted-malware-with-minimal-samples/>.